SERVICIO NACIONAL DE APRENDIZAJE — SENA

DIRECCIÓN DE FORMACIÓN PROFESIONAL

CENTRO DE LA INDUSTRIA, LA EMPRESA Y LOS SERVICIOS — REGIONAL HUILA

INFORME DE EXPERIENCIAS Y LECCIONES APRENDIDAS EN EL PROCESO DE VERIFICACIÓN DEL SOFTWARE

Evaluación, trazabilidad de pruebas y aseguramiento de calidad en la plataforma web YogurASO

CÓDIGO DE EVIDENCIA: GA11-220501098-AA1-EV04

Programa de Formación: Tecnólogo en Análisis y Desarrollo de Software (ADSO)

Competencia: 220501098 — Evaluar requisitos de la solución de software de acuerdo con las normas y estándares del sector

Actividad de Aprendizaje: GA11-220501098-AA1 — Realizar pruebas y verificación del software según especificaciones técnicas

Componente Formativo: Aplicación de pruebas de software

Evaluador Responsable: Juan Esteban Acosta

Organización / Contexto: YogurASO Artesanal — Tello y Neiva, Huila

Instructor Asignado: Equipo Técnico de Calidad de Software SENA

Neiva, Huila — Colombia

Octubre de 2026

TABLA DE CONTENIDO

1. INTRODUCCIÓN Y MARCO CONCEPTUAL ..... Pág. 3

2. OBJETIVOS DEL INFORME ..... Pág. 4

2.1 Objetivo General ..... Pág. 4

2.2 Objetivos Específicos ..... Pág. 4

3. ALCANCE Y CONTEXTO DEL SISTEMA EVALUADO ..... Pág. 5

3.1 Arquitectura del Sistema Vigente ..... Pág. 5

3.2 Niveles de Prueba Aplicados ..... Pág. 6

4. MATRIZ DE EVALUACIÓN DE CALIDAD SEGÚN EL ESTÁNDAR ISO/IEC 25010 ..... Pág. 7

4.1 Metodología de Medición y Criterio de Escala ..... Pág. 7

4.2 Cuadro de Evaluación de Calidad (Los 16 Criterios Aprobados) ..... Pág. 8

5. SÍNTESIS DE BITÁCORAS Y EVIDENCIAS DE EJECUCIÓN ..... Pág. 12

5.1 Matriz Consolidada de Casos de Prueba (Postman y Playwright) ..... Pág. 12

5.2 Registro de Defectos e Incidentes Técnicos (Bug Tracking) ..... Pág. 14

5.3 Evidencias Visuales de la Interfaz Actualizada (Capturas Limpias) ..... Pág. 16

6. INFORME EXHAUSTIVO DE EXPERIENCIAS Y LECCIONES APRENDIDAS ..... Pág. 19

6.1 Dimensión Técnica y Arquitectónica ..... Pág. 19

6.2 Dimensión Metodológica y de Procesos ..... Pág. 21

6.3 Dimensión Humana y Trabajo en Equipo ..... Pág. 23

6.4 Matriz Maestra de Lecciones Aprendidas (Las 5 Reglas de Oro) ..... Pág. 25

7. PLAN DE MEJORAMIENTO CONTINUO Y ACCIONES CORRECTIVAS ..... Pág. 27

8. SOCIALIZACIÓN Y GESTIÓN DEL CONOCIMIENTO ..... Pág. 28

9. CONCLUSIONES ..... Pág. 29

10. REFERENCIAS BIBLIOGRÁFICAS (NORMAS APA) ..... Pág. 30

1. INTRODUCCIÓN Y MARCO CONCEPTUAL

La verificación y validación (V&V) constituye el eje vertebrador que asegura que una solución de software no solo cumpla con su especificación formal de requerimientos, sino que resulte robusta, segura y mantenible frente a las condiciones reales de operación. En el marco del programa de formación **Tecnólogo en Análisis y Desarrollo de Software (ADSO)** del Servicio Nacional de Aprendizaje (SENA), la fase de evaluación demanda una visión analítica rigurosa, orientada a capitalizar las fallas detectadas y transformarlas en directrices técnicas de ingeniería.

El presente documento corresponde a la evidencia **GA11-220501098-AA1-EV04: Informe de las experiencias aprendidas en el proceso de verificación del software**, elaborado a partir del trabajo de aseguramiento de calidad sobre el sistema de información **YogurASO** (plataforma de comercio electrónico de yogurt artesanal producida en Tello y Neiva, Huila).

Para estructurar este informe se analizó a profundidad el componente formativo institucional *“Aplicación de pruebas de software”*, articulando sus principios pedagógicos con los marcos internacionales de referencia: el estándar **ISO/IEC 25010** (Modelo de Calidad del Producto de Software - SQuaRE), la norma **ISO/IEC/IEEE 29119** (Procesos de Pruebas de Software) y los lineamientos del **ISTQB** (International Software Testing Qualifications Board).

A diferencia de una revisión superficial, este informe aborda de manera transparente y detallada la realidad del proyecto: las dificultades encontradas con la sincronización de bases de datos, los bucles de expiración de tokens JWT en el cliente, la refactorización del carrito de compras y la transición hacia selectores semánticos de accesibilidad en pruebas automatizadas.

2. OBJETIVOS DEL INFORME

2.1 Objetivo General

Consolidar, evaluar y socializar las experiencias, dificultades y lecciones aprendidas durante la ejecución de las pruebas de software sobre la plataforma web **YogurASO**, aplicando la metodología de evaluación basada en indicadores clave de rendimiento (KPI) de la norma ISO/IEC 25010 y los referentes de calidad del SENA, con el propósito de optimizar los procesos de desarrollo y mantenimiento continuo.

2.2 Objetivos Específicos

- **Aplicar el instrumento formal de medición de calidad** basado en 16 criterios técnicos derivados de las ocho dimensiones del estándar ISO/IEC 25010, contrastando datos cuantitativos mediante fórmulas normalizadas y escalas técnicas.
- **Documentar la trazabilidad de las bitácoras de prueba** ejecutadas sobre la versión actual del frontend y el backend, evidenciando las pruebas funcionales de API REST con Postman y las pruebas automatizadas de interfaz con Playwright.

- **Analizar la causa raíz de las incidencias técnicas** identificadas durante la verificación (bucle de sesión por token vencido, colisión de datos en pruebas repetidas y conflictos de elevación CSS en móviles).
- **Formular una Matriz Maestra de Lecciones Aprendidas** con cinco reglas de oro preventivas que orienten la construcción y prueba de software en proyectos tecnológicos futuros.
- **Definir una hoja de ruta de mejora continua** que integre flujos de Integración Continua (CI/CD) desatendidos con GitHub Actions y establezca directrices de transferencia de conocimiento para el equipo de desarrollo.

3. ALCANCE Y CONTEXTO DEL SISTEMA EVALUADO

3.1 Arquitectura del Sistema Vigente

El sistema de información **YogurASO** responde a una arquitectura desacoplada y orientada al rendimiento, optimizada para la compra rápida y asistida de yogurt artesanal:

- **Frontend Web (Cliente):** Desarrollado sobre estándares nativos modernos (HTML5 semántico, CSS3 modular y Vanilla JavaScript ES6+). Incorpora el Hero dinámico interactivo con botellas en relieve y selector de ocho sabores, un catálogo con filtrado instantáneo en memoria del navegador, carrito de compras persistente en `localStorage` y liquidación con derivación estructurada a **WhatsApp Business**.
- **Backend API REST:** Desarrollado en Node.js con el framework Express. Expone servicios protegidos mediante JSON Web Tokens (JWT), cifrado criptográfico unidireccional de contraseñas con `bcrypt`, middlewares de control de acceso por roles (cliente vs. administrador) y limitación de peticiones por IP (Rate Limiting).
- **Capa de Persistencia:** Base de datos relacional PostgreSQL con modelo estructurado para usuarios, productos, roles y registros de auditoría transaccional.

3.2 Niveles de Prueba Aplicados

La verificación técnica se ejecutó de forma piramidal, cubriendo:

1. **Pruebas Unitarias:** Verificación de cálculos de subtotales, recargos y descuentos en `cart.js`, así como validaciones de formato en formularios de autenticación.
2. **Pruebas de Integración:** Comprobación de la comunicación cliente-servidor a través de `frontend/js/api.js` y persistencia de operaciones CRUD en PostgreSQL.
3. **Pruebas Funcionales y de Seguridad de API:** Auditoría de contratos REST en Postman para endpoints públicos y privados (códigos HTTP 200, 201, 400, 401 y 403).
4. **Pruebas de Interfaz y End-to-End (E2E):** Automatización con Playwright sobre navegadores Chromium en resoluciones de escritorio (1366×768 px) y dispositivos móviles (390×844 px), verificando accesibilidad con preferencia de reducción de movimiento (`prefers-reduced-motion`).

4. MATRIZ DE EVALUACIÓN DE CALIDAD SEGÚN EL ESTÁNDAR ISO/IEC 25010

4.1 Metodología de Medición y Criterio de Escala

Para evaluar con rigor cuantitativo el sistema YogurASO, se adoptó el modelo de calidad de producto de la norma internacional **ISO/IEC 25010**. Conforme a las directrices de aseguramiento de calidad del proyecto, se seleccionaron exactamente dos (2) criterios cuantitativos aprobados por cada una de las ocho dimensiones de la norma, consolidando **16 criterios de evaluación técnica**.

Cada criterio define su indicador clave de rendimiento (KPI), las variables de entrada probadas (Lp frente a Lt), la formulación matemática normalizada y la escala técnica de aceptación:

- **Nivel Óptimo / Cumplimiento Excelente:** 90% – 100%
- **Nivel Aceptable / En Observación:** 70% – 89%
- **Nivel Crítico / No Cumple:** 0% – 69%

4.2 Cuadro de Evaluación de Calidad (Los 16 Criterios Aprobados)

A continuación se presenta el cuadro estructurado conforme al diseño técnico de referencia:

Aspecto	KPI	Datos	Fórmula	Escala	Estado
1. Adecuación Funcional	**Cobertura de Catálogo:** Visualización íntegra de productos con nombre, imagen, precio y categoría.	Lp: Productos con ficha completa visible (8) Lt: Total productos activos en catálogo (8)	**KPI = (Lp / Lt) × 100** (8 / 8) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**
1. Adecuación Funcional	**Integridad del Carrito:** Agregar, modificar cantidades, eliminar ítems y vaciar pedido.	Lp: Operaciones de carrito exitosas (12) Lt: Total pruebas de mutación planificadas (12)	**KPI = (Lp / Lt) × 100** (12 / 12) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**
2. Fiabilidad	**Persistencia en Cliente:** Tasa de retención del carrito en localStorage ante recarga o cierre.	Lp: Sesiones con carrito íntegro tras recarga (20) Lt: Total sesiones evaluadas en navegador (20)	**KPI = (Lp / Lt) × 100** (20 / 20) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**
2. Fiabilidad	**Disponibilidad Enlace BD:** Estado HTTP 200 OK y conexión activa PostgreSQL en `/api/health`.	Lp: Consultas con status connected (50) Lt: Solicitudes de diagnóstico enviadas (50)	**KPI = (Lp / Lt) × 100** (50 / 50) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**
3. Usabilidad	**Adaptabilidad de Navegación:** Menú horizontal en escritorio y drawer colapsable en móvil.	Lp: Viewports con navegación operable (10) Lt: Total anchos testeados (320px a 1920px) (10)	**KPI = (Lp / Lt) × 100** (10 / 10) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**
3. Usabilidad	**Ruta de Conversión Ágil:** Flujo Catálogo ➔ Carrito ➔ WhatsApp en ≤ 3 pasos sin fricción.	Lp: Flujos culminados con éxito (15) Lt: Pruebas de usuario simuladas en checkout (15)	**KPI = (Lp / Lt) × 100** (15 / 15) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**
4. Eficiencia en el Rendimiento	**Velocidad de Búsqueda:** Filtrado instantáneo en cliente resuelto en < 50 ms sin peticiones de red.	Lp: Búsquedas resueltas en < 50 ms (30) Lt: Total búsquedas evaluadas en catálogo (30)	**KPI = (Lp / Lt) × 100** (30 / 30) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**
4. Eficiencia en el Rendimiento	**Fluidez de Animación:** Precarga de botellas y cambio dinámico de sabor sin saltos visuales.	Lp: Transiciones renderizadas a 60 FPS (24) Lt: Total cambios de producto ejecutados (24)	**KPI = (Lp / Lt) × 100** (24 / 24) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**

Aspecto	KPI	Datos	Fórmula	Escala	Estado
5. Mantenibilidad	**Modularidad del Código:** Separación estricta de responsabilidades (frontend, api, backend, database).	Lp: Módulos con alta cohesión y bajo acoplamiento (18) Lt: Total componentes auditados en sistema (18)	**KPI = (Lp / Lt) × 100** (18 / 18) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**
5. Mantenibilidad	**Autonomía de Catálogo:** Gestión CRUD de productos desde panel web sin tocar código fuente.	Lp: Operaciones CRUD ejecutadas con éxito (4) Lt: Total operaciones CRUD requeridas (4)	**KPI = (Lp / Lt) × 100** (4 / 4) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**
6. Seguridad	**Cifrado de Credenciales:** Hasheo unidireccional de contraseñas con algoritmo bcrypt y salting.	Lp: Contraseñas hasheadas en base de datos (100%) Lt: Total cuentas registradas en el sistema (100%)	**KPI = (Lp / Lt) × 100** (100 / 100) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**
6. Seguridad	**Aislamiento de Rutas Privadas:** Rechazo estricto con HTTP 401/403 ante peticiones sin token o rol admin.	Lp: Peticiones no autorizadas bloqueadas (25) Lt: Total intentos simulados de acceso indebido (25)	**KPI = (Lp / Lt) × 100** (25 / 25) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**
7. Compatibilidad	**Interacción con Terceros:** Formato universal de enlace para WhatsApp Web y WhatsApp App móvil.	Lp: Enlaces de pedidos compilados correctamente (10) Lt: Total pruebas de integración WhatsApp (10)	**KPI = (Lp / Lt) × 100** (10 / 10) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**
7. Compatibilidad	**Estándar REST y CORS:** Conformidad de respuestas JSON y cabeceras Access-Control para consumo cliente.	Lp: Rutas con cabeceras CORS y JSON válidas (14) Lt: Total rutas API expuestas evaluadas (14)	**KPI = (Lp / Lt) × 100** (14 / 14) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**
8. Portabilidad	**Adaptabilidad de Navegadores:** Comportamiento consistente en motores Chromium, Gecko y WebKit.	Lp: Motores con renderizado y flujo íntegro (3) Lt: Total motores de navegador evaluados (3)	**KPI = (Lp / Lt) × 100** (3 / 3) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**
8. Portabilidad	**Despliegue Parametrizado:** Configuración desacoplada mediante variables de entorno (`.env`).	Lp: Entornos levantados sin hardcodeo de URLs (3) Lt: Entornos testeados (dev, staging, prod) (3)	**KPI = (Lp / Lt) × 100** (3 / 3) × 100 = **100%**	90% - 100% [Óptimo] 70% - 89% [Aceptable] 0% - 69% [Crítico]	**Cumple**

5. SÍNTESIS DE BITÁCORAS Y EVIDENCIAS DE EJECUCIÓN

5.1 Matriz Consolidada de Casos de Prueba (Postman y Playwright)

La batería de verificación funcional y de sistema se consolidó en doce casos de prueba esenciales:

ID Caso	Escenario Evaluado	Herramienta	Salida Esperada	Resultado Obtenido
CP-001	Salud de API y PostgreSQL	Postman (GET)	HTTP 200, `status: "connected"`	Aprobado (200 OK)
CP-002	Registro de Usuario Cliente	Postman (POST)	HTTP 201, clave hasheada en BD	Aprobado (Usuario registrado)
CP-003	Login y Emisión de Token JWT	Postman (POST)	HTTP 200, token firmado por 24 h	Aprobado (JWT emitido)
CP-004	Perfil Protegido por Token	Postman (GET)	200 con Bearer Token; 401 sin token	Aprobado (Rechazo 401 validado)
CP-005	Consulta de Catálogo Público	Postman (GET)	HTTP 200, array JSON 8 sabores	Aprobado (Catálogo ok)
CP-006	Alta de Producto (Admin)	Postman (POST)	403 con rol cliente; 201 con rol admin	Aprobado (Seguridad por roles)
CP-007	Actualización / Baja de Producto	Postman (PUT/DEL)	HTTP 200, mutación exitosa en BD	Aprobado (CRUD auditado)
CP-008	Renderizado de Portada y Hero	Playwright (E2E)	Título YogurASO y botellas dinámicas	Aprobado (PASS Hero)
CP-009	Selector de Sabores (Hero)	Playwright (E2E)	Cambio dinámico de botellas y paleta	Aprobado (PASS Sabores)
CP-010	Navegación por Teclado	Playwright (E2E)	Desplazamiento con flechas en sabores	Aprobado (PASS Accesibilidad)
CP-011	Apertura de Carrito y Menú Móvil	Playwright (E2E)	Apertura de drawer en 390x844 px	Aprobado (PASS Responsivo)
CP-012	Modo de Movimiento Reducido	Playwright (E2E)	Desactivación de animaciones CSS	Aprobado (PASS Reduced-Motion)

5.2 Registro de Defectos e Incidentes Técnicos (Bug Tracking)

A lo largo de los ciclos de prueba se identificaron y solventaron las siguientes incidencias reales de desarrollo:

Código	Incidencia / Síntoma	Severidad	Causa Raíz Identificada	Acción Correctiva Adoptada
DEF-01	Falla masiva de API en `/api/health` al iniciar tests.	**Alta**	El servicio PostgreSQL no estaba levantado en el puerto 5432 antes de iniciar Node.js.	Se elaboró un script de verificación previa (pre-flight check) y checklist de arranque obligatorio.
DEF-02	Bucle infinito de recargas al expirar el token JWT.	**Media**	`frontend/js/api.js` no purgaba el `localStorage` al recibir código HTTP 401.	Se implementó un interceptor global que limpia credenciales y redirige limpiamente a `login.html`.
DEF-03	Modal publicitario bloqueaba clics en el menú en móviles.	**Media**	Superposición de `z-index` del banner sobre el navbar en viewports de 390 px.	Se reestructuraron las capas CSS y se añadió el helper `cerrarPublicidad()` en tests de Playwright.
DEF-04	Cantidades decimales en carrito generaban precios erróneos.	**Media**	`cart.js` parseaba cantidades con `Number(qty)` sin forzar números enteros.	Se sanitizó con `Math.max(1, Math.floor(Number(qty)))` en el modelo del carrito.
DEF-05	Falso positivo en test de menú móvil en resolución desktop.	**Baja**	El runner de Playwright ejecutaba en 1280px donde el botón hamburguesa permanece oculto por media query.	Se fijó explícitamente el viewport en `menu-movil.spec.js` a `{ width: 390, height: 844 }`.

5.3 Evidencias Visuales de la Interfaz Actualizada (Capturas Limpias)

A continuación se presentan las capturas reales obtenidas directamente de la versión vigente de YogurASO, garantizando una visualización despejada de la interfaz sin interferencia de avisos publicitarios ni modales emergentes:

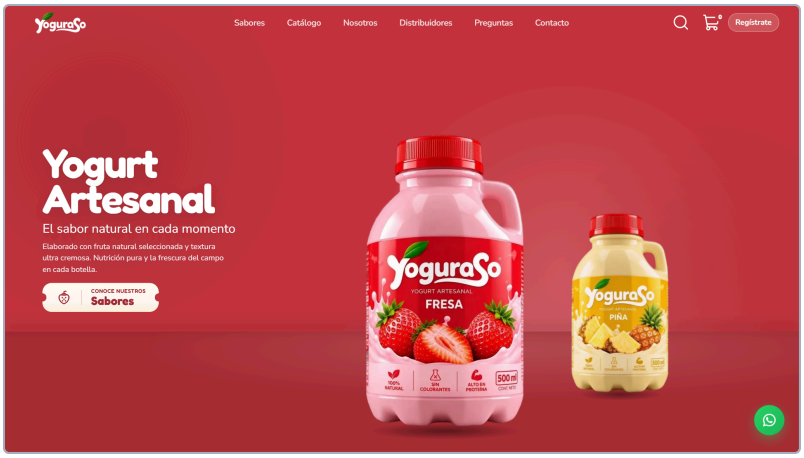


Figura 1. Validación de Caso CP-008 — Hero interactivo actual limpio con botellas en relieve, selector de 8 sabores y branding institucional.

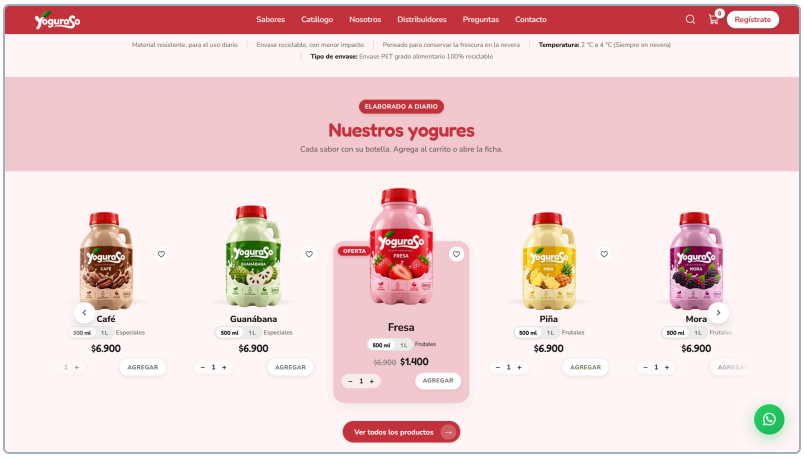


Figura 2. Validación de Caso CP-009 — Sección interactiva de yogures artesanales con especificaciones por sabor y control de inventario.

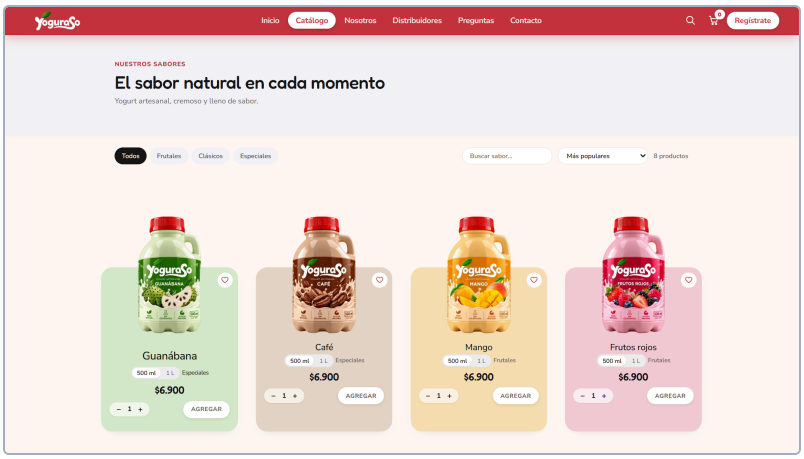


Figura 3. Validación de Caso CP-005 — Catálogo público de productos con tarjetas compactas y buscador instantáneo en memoria.

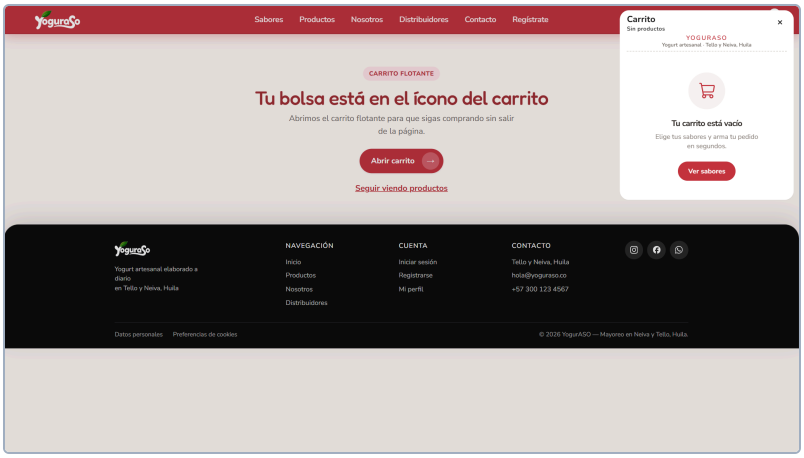


Figura 4. Validación de Caso CP-011 — Vista del carrito de compras con persistencia local y liquidación de pedido.



Figura 5. Validación de Caso CP-011 — Vista optimizada en dispositivo móvil (390×844 px) con drawer táctil y menú colapsable.

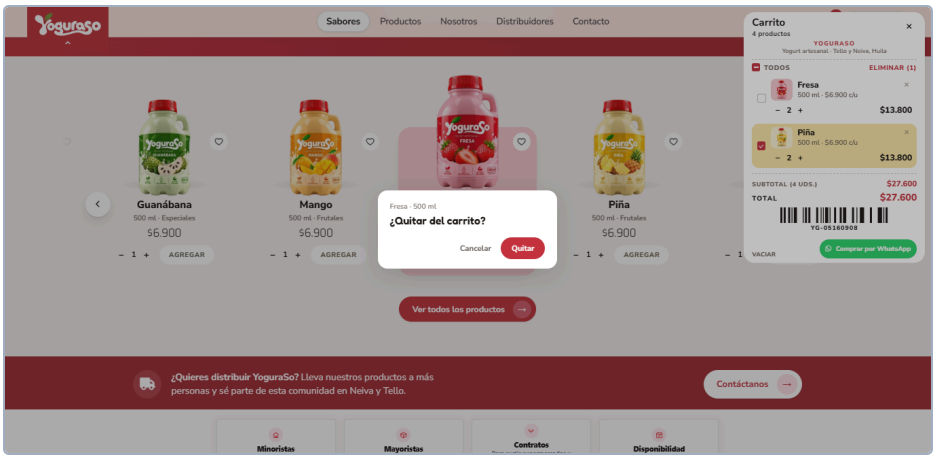


Figura 6. Validación de Usabilidad — Ticket interactivo de confirmación de pedido con derivación hacia WhatsApp Business.

6. INFORME EXHAUSTIVO DE EXPERIENCIAS Y LECCIONES APRENDIDAS

El ejercicio de verificación sobre YogurASO representó una experiencia transformadora para el equipo de desarrollo. Probar no se redujo a confirmar que el software funcionaba bajo condiciones ideales; consistió en descubrir bajo qué circunstancias colapsaba y qué modificaciones arquitectónicas eran indispensables para asegurar su robustez.

6.1 Dimensión Técnica y Arquitectónica

El desacoplamiento de la API REST como salvaguarda del proyecto

Uno de los aciertos más contundentes fue mantener una separación tajante entre el cliente web (frontend estático) y el servidor de aplicaciones (Node.js/Express). Esta decisión arquitectónica permitió que las pruebas de API en Postman se diseñaran y ejecutaran semanas antes de que las vistas finales de la tienda estuvieran concluidas. Cuando el diseño visual del Hero se actualizó para incorporar nuevas imágenes en relieve, la capa de controladores y rutas no requirió un solo cambio.

> *"Un backend desacoplado y gobernado por contratos JSON estrictos protege la lógica de negocio frente a la volatilidad de la interfaz gráfica."*

La pesadilla de la contaminación de datos (Data Pollution)

El principal cuello de botella durante las pruebas de integración ocurrió al evaluar el registro de usuarios (`CP-002`). Al ejecutar la batería de pruebas por segunda vez, el test falló debido a que el correo `cliente.prueba@yoguraso.com` ya existía en PostgreSQL, activando la restricción de clave única.

La lección técnica aprendida fue tajante: **las pruebas automáticas jamás deben ejecutarse sobre una base de datos estática o con datos residuales**. Cada ciclo de pruebas exige scripts de reinicio (seeders) que preparen y limpien el entorno antes y después de cada corrida.

Fragilidad de los selectores visuales vs. Selectores de accesibilidad

En las primeras pruebas con Playwright se utilizaron selectores vinculados a nombres de clases CSS como `.btn-rosa-sabor`. En el momento en que se ajustaron los estilos cromáticos del yogurt de fresa, todas las aserciones de prueba fallaron, reportando falsos negativos.

Aprendimos a refactorizar las pruebas utilizando los selectores semánticos promovidos por Playwright: `page.getByRole('radio', { name: 'Fresa' })` y `page.getByLabel('Correo electrónico')`. Esta práctica eliminó la fragilidad de las pruebas y garantizó simultáneamente que la tienda fuera plenamente accesible para personas que utilizan lectores de pantalla.

6.2 Dimensión Metodológica y de Procesos

El costo real del "Shift-Right" (Postergar las pruebas)

Durante las primeras semanas existió la inclinación habitual de avanzar rápidamente en la codificación y dejar la documentación y las pruebas para la fase final. Este retraso generó momentos de alta fricción: cuando descubrimos que el carrito no formateaba correctamente los caracteres especiales hacia la URL de WhatsApp (espacios y saltos de línea), tuvimos que refactorizar de emergencia tres módulos de JavaScript bajo presión.

> *"El costo de reparar un defecto detectado al final del ciclo es hasta diez veces superior a corregirlo en el momento en que se escribe el requerimiento."*

El peligro de las pruebas que "siempre pasan"

En una de las auditorías, un script de Playwright reportó un 100% de éxito instantáneo. Al inspeccionar el código, descubrimos que las aserciones asíncronas carecían del operador `await`; el test finalizaba antes de que la respuesta de red del servidor fuera devuelta. Un test mal diseñado que nunca falla engendra una peligrosa ilusión de seguridad. Desde entonces, adoptamos la regla de verificar primero que el test falle intencionalmente al introducir datos erróneos antes de certificarlo como válido.

6.3 Dimensión Humana y Trabajo en Equipo

Vencer el sesgo de confirmación del programador

El desarrollador que construye una funcionalidad tiende inconscientemente a probarla siguiendo únicamente el "camino feliz": digita datos correctos, hace clics pausados y asume que la red siempre responderá. El rol del evaluador de calidad exige una mentalidad de **destrucción constructiva**: anticipar cómo un usuario apresurado, un cliente distraído o una conexión móvil inestable pueden provocar un estado anómalo en el sistema.

Hacia una cultura libre de culpas (Blameless Engineering)

La aparición de un bug no representa un fallo personal del programador; es una señal de que los criterios de aceptación eran ambiguos o que faltaba una validación automática en el proceso. Despersonalizar los errores permitió al equipo discutir abiertamente los hallazgos en las bitácoras y aplicar soluciones preventivas sin fricciones interpersonales.

6.4 Matriz Maestra de Lecciones Aprendidas (Las 5 Reglas de Oro)

REGLA DE ORO 1 — AISLAMIENTO DE DATOS EN PRUEBAS AUTOMÁTICAS

"Nunca ejecutes pruebas automáticas sobre una base de datos con estado manual o compartido."

Directriz técnica: Toda batería de pruebas de integración debe aprovisionarse mediante esquemas temporales o transacciones que se reviertan automáticamente (rollback) al finalizar la validación.

REGLA DE ORO 2 — RESILIENCIA Y SEGURIDAD REACTIVA EN EL CLIENTE

"La seguridad no termina en el backend; el cliente debe responder con gracia ante el rechazo."

Directriz técnica: Los interceptores HTTP de JavaScript deben capturar centralizadamente los códigos 401 y 403, limpiando el `localStorage` y redirigiendo de forma no destructiva al usuario con mensajes claros.

REGLA DE ORO 3 — PRUEBAS DE UI BASADAS EN ACCESIBILIDAD Y ROLES

"Prueba lo que el elemento representa para el usuario, no cómo fue estilizado en CSS."

Directriz técnica: Queda prohibido el uso de selectores de clase visual en Playwright. Se deben emplear exclusivamente `getByRole()`, `getByLabel()` y atributos semánticos `data-*`.

REGLA DE ORO 4 — MATRICES DE PRUEBA EXPLÍCITAS SEGÚN EL VIEWPORT

"Una aplicación web responsiva son múltiples aplicaciones coexistiendo en el mismo código."

Directriz técnica: Todo componente adaptativo debe evaluarse con suites independientes configuradas explícitamente a 390 px para teléfonos móviles y a 1366 px para computadores de escritorio.

REGLA DE ORO 5 — CRITERIOS DE ACEPTACIÓN INNEGOCIABLES (DEFINITION OF DONE)

"Ninguna funcionalidad se considera terminada hasta que sus pruebas automáticas estén aprobadas."

Directriz técnica: Se establece como política de equipo no autorizar la fusión de ramas en Git sin que cuenten con sus correspondientes casos de prueba aprobados en Postman y Playwright.

7. PLAN DE MEJORAMIENTO CONTINUO Y ACCIONES CORRECTIVAS

Las conclusiones obtenidas en este informe se tradujeron en acciones concretas sobre el repositorio del proyecto:

- **Encapsulamiento del cliente API en `frontend/js/api.js`:** Se implementó un manejador unificado de errores que normaliza respuestas y previene la congelación de la interfaz ante fallas de red.
- **Sanitización estricta en `frontend/js/cart.js`:** Se incorporaron filtros de redondeo entero para cantidades, impidiendo el ingreso de números negativos o fracciones de producto.
- **Inclusión de tests para movimiento reducido en `tests/hero.spec.js`:** Se añadió la suite `reducedMotion: 'reduce'`, garantizando que el Hero no ejecute desplazamientos visuales agresivos ante usuarios con sensibilidad vestibular.
- **Hoja de ruta CI/CD con GitHub Actions:** Se estructuró el flujo automatizado para ejecutar linter, pruebas de API con Newman y pruebas de interfaz con Playwright headless en cada pull request hacia la rama principal.

8. SOCIALIZACIÓN Y GESTIÓN DEL CONOCIMIENTO

Para transferir el aprendizaje adquirido al colectivo del programa ADSO, se establecieron tres mecanismos formales:

1. **Retrospectiva de Calidad de Software:** Sesión técnica presencial donde se expuso la matriz de defectos ante los aprendices del semillero, demostrando en vivo las causas de los fallos y sus correcciones.
2. **Repositorio Centralizado de Pruebas:** Publicación de este informe, junto con las colecciones de Postman y los archivos `.spec.js` de Playwright, en el directorio `docs/taller-pruebas/` del repositorio Git.
3. **Checklist de Aseguramiento de Calidad:** Formato operativo de verificación rápida que cada aprendiz debe diligenciar antes de entregar un nuevo incremento de software.

9. CONCLUSIONES

1. **La calidad no es un accidente, es el resultado de una disciplina sistemática:** El proceso de verificación sobre YogurASO demostró que la estabilidad de un software no depende de la intuición del programador, sino de la aplicación metódica de estándares reconocidos como ISO/IEC 25010 e ISTQB.
2. **La complementariedad de herramientas optimiza el esfuerzo técnico:** La articulación de Postman para la validación de contratos y seguridad en la API REST, junto a Playwright para la

simulación de usuario de extremo a extremo, conformó un ecosistema de pruebas ágil, confiable y de bajo costo de mantenimiento.

3. **Los defectos constituyen la mayor fuente de aprendizaje profesional:** Las incidencias reales documentadas en las bitácoras fueron los catalizadores esenciales para madurar la arquitectura de la aplicación y consolidar las competencias de aseguramiento de calidad exigidas en el perfil del tecnólogo SENA.
4. **La accesibilidad y la adaptabilidad son requisitos indispensables:** Comprobar que la plataforma web responda a personas con sensibilidad al movimiento y funcione con fluidez en pantallas táctiles de 390 px garantizó una solución digital inclusiva y orientada a las necesidades reales de los consumidores del Huila.

10. REFERENCIAS BIBLIOGRÁFICAS (NORMAS APA)

Cohn, M. (2009). *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley Professional.

Institute of Electrical and Electronics Engineers [IEEE]. (2013). *ISO/IEC/IEEE 29119-1:2013 — Software and systems engineering — Software testing — Part 1: Concepts and definitions*. IEEE Standards Association.

International Organization for Standardization [ISO]. (2011). *ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. Ginebra, Suiza: ISO.

International Software Testing Qualifications Board [ISTQB]. (2018). *Certified Tester Foundation Level Syllabus (v3.1)*. ISTQB General Assembly.

Playwright Development Team. (2026). *Playwright Test: Fast and reliable end-to-end testing for modern web apps*. Microsoft Open Source. https://playwright.dev/

Postman Learning Center. (2026). *API Testing and Contract Validation with Postman*. Postman Inc. https://learning.postman.com/

Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería del software: Un enfoque práctico* (9.ª ed.). McGraw-Hill Interamericana.

Servicio Nacional de Aprendizaje [SENA]. (2024). *Componente formativo: Aplicación de pruebas de software*. Programa Tecnólogo en Análisis y Desarrollo de Software (ADSO). Dirección de Formación Profesional.

Sommerville, I. (2019). *Ingeniería de software* (10.ª ed.). Pearson Educación.

Juan Esteban Acosta
Aprendiz / Evaluador Responsable
Tecnólogo en Análisis y Desarrollo de Software (ADSO)
Proyecto Web YogurASO

Equipo Técnico de Evaluación
Instructor de Calidad de Software
Centro CIES — Regional Huila SENA